

A1:PROPOSAL OF QUESTION

Can we identify the groups in our Churn dataset based on their usage of our Bandwidth_GB_Year services, the monthly charges, and the tenure using K-Means clustering?

A2: DEFINED GOAL

The goal of this analysis is to identify segments in our telecommunication customer base using K-means clustering in order to formulate better strategic business decisions around the company's Bandwidth_GB_Year, MonthlyCharge, and customers' longevity, represented by the tenure, which will result in better profits for the telecommunication company in the long run.

B1: EXPLANATION OF THE CLUSTERING TECHNIQUE

For the purpose of this analysis, I have chosen the K-Means Clustering technique to analyze the Churn dataset.

K-Means clustering is an unsupervised machine learning algorithm that groups unlabeled datasets into different clusters based on their similarity. K-means partitions a dataset into a predefined number of clusters (K) by minimizing the within-cluster variance.

According to Whitfield, Brennan (2025, March 13), K-means clustering is an unsupervised, distance-based clustering algorithm where data points that are close to each other are grouped in a given number of clusters/groups.

K-means analyzes the dataset by

- Randomly initializing K centroids, each representing the center of a cluster.
- Assigning each data point in the dataset to the nearest centroid, forming K clusters

- The centroids are then updated to be the mean of the data points in their respective clusters.
- The process is iterated until the centroids no longer change significantly, indicating convergence.
- The selected features are scaled to ensure uniformity in distance calculations.
- K-Means uses Euclidean distance to evaluate similarity between data points.
- The K-means algorithm detects natural groupings in the data by placing similar customers in the same cluster based on feature proximity.
- Each cluster formed represents a unique segment of the dataset.

The expected outcome from the model is that the model will output

- A cluster label for each data point.
- The coordinates of each cluster centroid in the feature space.
- The model will give insights into customer segmentation that will allow us to identify distinct patterns.

B2: SUMMARY OF THE TECHNIQUE ASSUMPTION

K-Means clustering assumes that all clusters have similar variance.

B3: PACKAGES OR LIBRARIES LIST

To undertake the analysis, I will be utilizing the Python tool. Python is an open-source, general-purpose programming language with many library packages that can be used and shared with others. I utilized the Python Programming Language because of its flexibility and extensive library collections, which are valuable for performing complex calculations and analysis. For Python, I will be using Python version 3.12.2

The following Python libraries will be utilized to perform the various functionalities:

1. Pandas

Pandas will be utilized to import the data from the CSV file into a structured dataframe, conduct exploratory data analysis, and manipulate the data.

2. Numpy

Numpy will be used for data manipulation. For the purpose of the K-means analysis, NumPy will be utilized for matrix operations, data transformations, and ensuring compatibility with libraries that require data in array format

3. Matplotlib.pyplot

Matplotlib will be used for data visualization. The visualization includes a line chart that is utilized to identify an elbow in the elbow method, so as to determine the optimal number of clusters (k) for K-Means clustering.

4. Seaborn

Seaborn will be used for data visualization in exploratory data analysis. The visualizations include boxplots, scatterplots

5. StandardScaler

The StandardScaler function from the library Sklearn. preprocessing will be used to standardize the features by removing the mean and scaling to unit variance. The StandardScaler is used to normalize numerical features by centering them around the mean and scaling to unit variance. This ensures that all features contribute equally to the distance calculations used in K-Means clustering, which is sensitive to varying feature scales.

6. KMeans

KMeans function from Sklearn. Cluster will be utilized to perform the unsupervised clustering.

KMeans groups data into a specified number of clusters (k) by minimizing the within-cluster sum of squares (WCSS) and assigning each point to the nearest cluster centroid.

7. Silhouette_score

Silhouette_score function from sklearn. Metrics will be utilized to evaluate the effectiveness of the clustering. The Silhouette_score calculates how well each data point fits within its assigned cluster compared to others, helping determine the optimal number of clusters. Higher scores indicate better-defined clusters.

C1: DATA PREPROCESSING

For this analysis, I chose to perform k-means clustering on the telecommunication churn dataset.

To be able to analyze the data, one key data preprocessing goal that is relevant to k-means clustering is data normalization (scaling). Normalization is essential so as to enable fair and accurate clustering using the k-means algorithm. Since k-means clustering relies on calculating distances between data points using Euclidean distance, features on different scales can disproportionately influence the clustering results. For instance, the Bandwidth_GB_Year variable was measured in large units and could dominate smaller-scaled features such as Tenure or MonthlyCharge, leading to biased clusters. To address this, the dataset was scaled using StandardScaler, which standardizes features by removing the mean and scaling to unit variance. This ensures that each feature contributes equally to the clustering process. The scaled data is then visualized to verify that the feature distribution was appropriately adjusted.

C2: DATA SET VARIABLES

After performing the data exploration, the following variables that had continuous data type were utilized for the analysis:

1. Tenure

The tenure represents the number of months the customer has stayed with the provider. The variable Tenure has a continuous datatype.

2. MonthlyCharge

The MonthlyCharge represents the amount charged to the customer. The variable MonthlyCharge has a continuous datatype.

3. Bandwidth_GB_Year

The Bandwidth_GB_Year represents the average amount of data used in GB in a year by the customer. The variable Bandwidth_GB_Year has a continuous datatype.

The variables were selected to perform the K-Means clustering since the K-Means clustering machine learning method requires that the variables have a continuous data type.

C3:STEPS FOR ANALYSIS

1. Import the necessary libraries

I started by importing the Python libraries needed for data manipulation, visualization, preprocessing, clustering, and evaluation. These include Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn modules such as StandardScaler for feature scaling, KMeans for clustering, and Silhouette_score for evaluating cluster quality.

```
#Importing libraries needed for the analysis
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

2. Load the dataset

I loaded the dataset into a Pandas Dataframe called Churn_df to begin the analysis.

```
#Loading the data
#File path variables

Local_path = 'C:/Users/beatr/OneDrive/Desktop/WGU/D212/Data/'

InputFilePath = Local_path + 'churn_clean.csv'
OutputFilePath = Local_path + 'churn_cleaned_df.csv'
Scaledfilepath = Local_path + 'scaled_df.csv'

Churn_df = pd.read_csv(InputFilePath)
```

3. Data exploration

I explored the data to identify the features needed for the analysis. To understand the shape, data types, and correlation among the variables.

```
#Initial data exploration (Understanding the data)
pd.set_option('display.max_columns', None)
Churn_df.head()
```

```
In [4]: ► Churn_df.shape
```

```
Out[4]: (10000, 50)
```

```
Churn_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 50 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   CaseOrder                             10000 non-null  int64
1   Customer_id                           10000 non-null  object
2   Interaction                             10000 non-null  object
3   UID                                    10000 non-null  object
4   City                                    10000 non-null  object
5   State                                  10000 non-null  object
6   County                                 10000 non-null  object
7   Zip                                    10000 non-null  int64
8   Lat                                    10000 non-null  float64
9   Lng                                    10000 non-null  float64
10  Population                             10000 non-null  int64
11  Area                                    10000 non-null  object
12  TimeZone                               10000 non-null  object
13  Job                                     10000 non-null  object
14  Children                               10000 non-null  int64
15  Age                                    10000 non-null  int64
16  Income                                 10000 non-null  float64
17  Marital                               10000 non-null  object
18  Gender                                 10000 non-null  object
19  Churn                                  10000 non-null  object
20  Outage_sec_perweek                     10000 non-null  float64
21  Email                                  10000 non-null  int64
22  Contacts                               10000 non-null  int64
23  Yearly_equip_failure                   10000 non-null  int64
24  Techie                                 10000 non-null  object
25  Contract                               10000 non-null  object
26  Port_modem                             10000 non-null  object
27  Tablet                                 10000 non-null  object
28  InternetService                        7871 non-null   object
29  Phone                                  10000 non-null  object
30  Multiple                               10000 non-null  object
31  OnlineSecurity                         10000 non-null  object
32  OnlineBackup                           10000 non-null  object
33  DeviceProtection                       10000 non-null  object
34  TechSupport                            10000 non-null  object
35  StreamingTV                            10000 non-null  object
36  StreamingMovies                        10000 non-null  object
37  PaperlessBilling                       10000 non-null  object
38  PaymentMethod                          10000 non-null  object
39  Tenure                                 10000 non-null  float64
40  MonthlyCharge                          10000 non-null  float64
41  Bandwidth_GB_Year                      10000 non-null  float64
42  Item1                                  10000 non-null  int64
43  Item2                                  10000 non-null  int64
44  Item3                                  10000 non-null  int64
45  Item4                                  10000 non-null  int64
46  Item5                                  10000 non-null  int64
47  Item6                                  10000 non-null  int64
48  Item7                                  10000 non-null  int64
49  Item8                                  10000 non-null  int64
dtypes: float64(7), int64(16), object(27)
memory usage: 3.8+ MB
```

```
#Analysing the Correlation /Relationship analysis
Continuous_columns = Churn_df.select_dtypes(include=['float64', 'int64']).columns.tolist()
Cor_data = Churn_df[Continuous_columns]

def Generate_heatmap_graph(corr, chart_title, mask_uppertri=False):
    """ Based on features, generate a correlation matrix """
    mask = np.zeros_like(corr, dtype=bool)
    mask[np.triu_indices_from(mask)] = mask_uppertri
    fig, ax = plt.subplots(figsize=(20, 20))
    sns.heatmap(corr,
                mask=mask,
                square=True,
                annot=True,
                annot_kws={'size': 12, 'rotation': 0, 'weight': 'bold'},
                cmap=plt.get_cmap('viridis'),
                linewidths=.1,
                ax=ax)
    ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha='right') # Rotate x-axis labels
    plt.title(chart_title, fontsize=32)
    plt.show()
var_corr = round(Cor_data.corr(), 2)
Generate_heatmap_graph(var_corr, chart_title='Churn_df Correlation Heatmap', mask_uppertri=True)
```

!

4. Drop irrelevant variables

I dropped the variables that are not needed for the clustering analysis. This helps reduce noise in the data and ensures the model focuses only on the most meaningful features.

```
# Select the features that have continous data for clustering
features_df = Churn_df[['Tenure', 'MonthlyCharge', 'Bandwidth_GB_Year']]
```

5. Data cleaning

- I checked the data for any missing values and dropped the null values

```
#Checking for Nulls and missing data
print(features_df[features_df[features_df.columns[features_df.isna().any()]].columns].info())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Empty DataFrame
None

#Dropping any missing values
features_df = features_df.dropna()
```

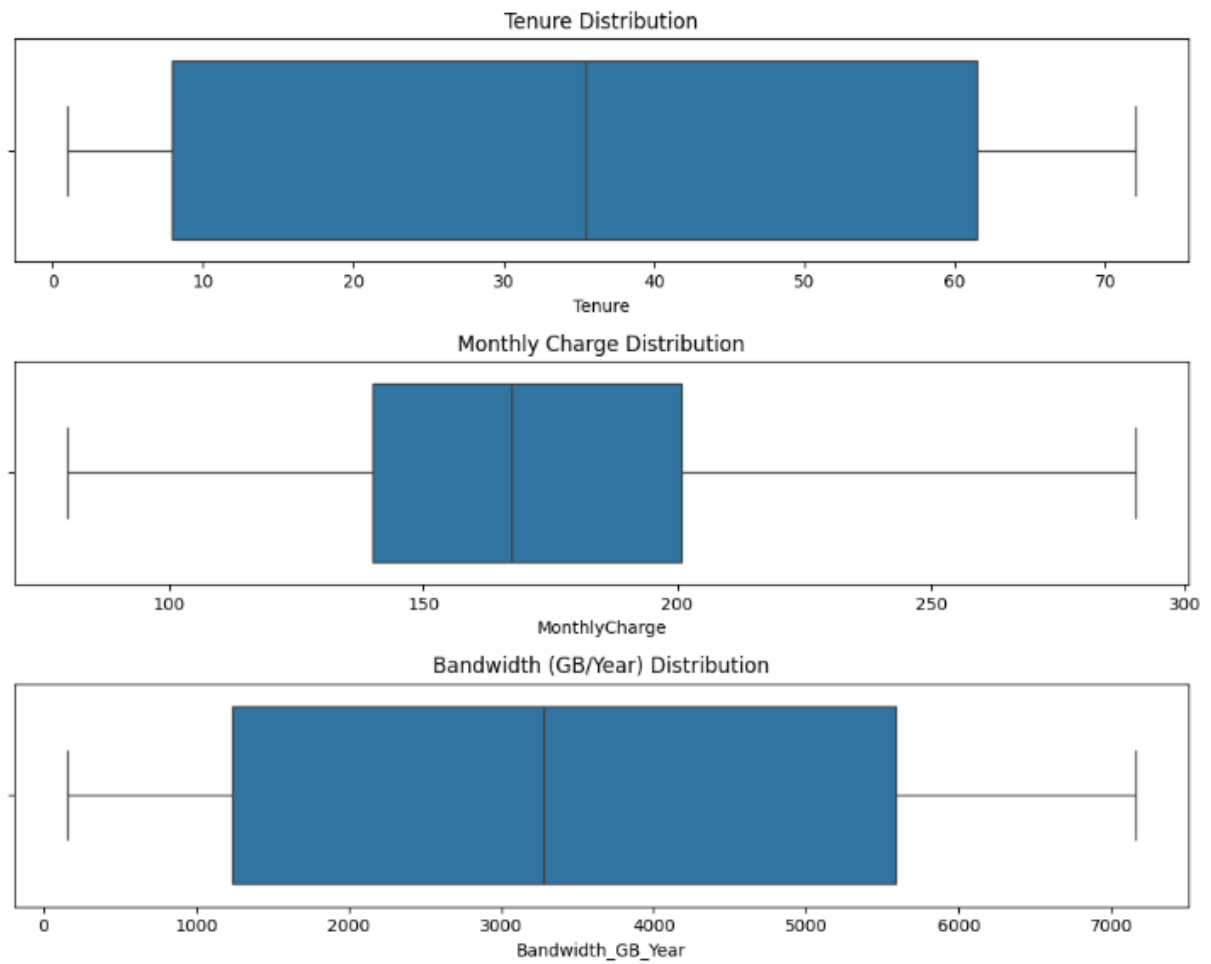
- I checked for outliers in the data and there were no outliers in the data for the selected features.

```
#Checking for Outliers in the dataset
# Set up the figure and axes
fig, axes = plt.subplots(3, 1, figsize=(10, 8))

# Flatten axes array for easy indexing
axes = axes.flatten()

# List of column names and titles
cols = ['Tenure', 'MonthlyCharge', 'Bandwidth_GB_Year']
titles = ['Tenure Distribution',
          'Monthly Charge Distribution', 'Bandwidth (GB/Year) Distribution']

# Create boxplots
for i, col in enumerate(cols):
    sns.boxplot(data=features_df, x=col, ax=axes[i])
    axes[i].set_title(titles[i])
    'Tenure Distribution'
# Adjust layout
plt.tight_layout()
plt.show()
```



- Feature scaling

I standardized the selected features to ensure they have equal weight in distance calculations.

Scaling is important for the K-Means algorithm to perform accurately.

```
In [11]: #Data Normalization/scaling the data
#Scaling the data using Standard scaler

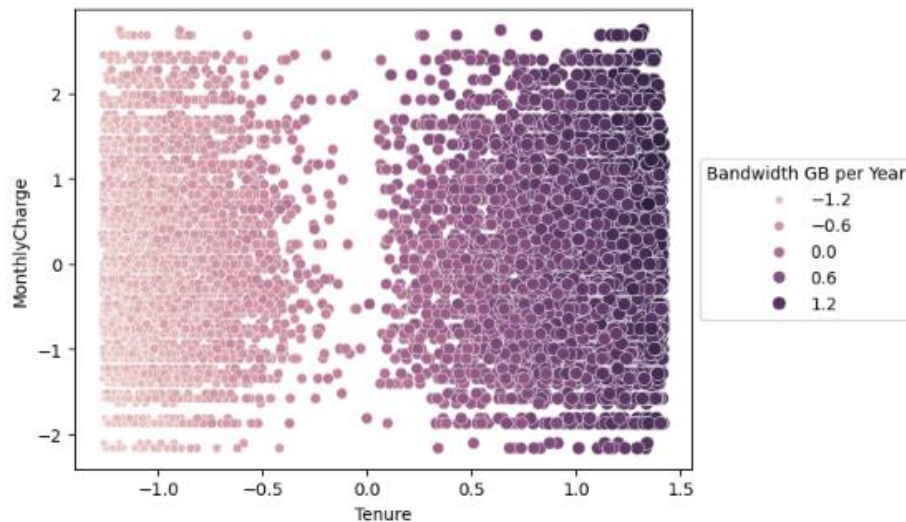
scaler = StandardScaler()
scaled_features = scaler.fit_transform(features_df)

#Converting the scaled data to a dataframe
scaled_df= pd.DataFrame(scaled_features ,columns=['Tenure', 'MonthlyCharge', 'Bandwidth_GB_Year'])
print(scaled_df.head())

print("Scaled data visualization")
sns.scatterplot(data=scaled_df, x='Tenure',y='MonthlyCharge' ,hue='Bandwidth_GB_Year', size= 'Bandwidth_GB_Year')
plt.legend( loc='center left',    bbox_to_anchor=(1, 0.5),title='Bandwidth GB per Year'
)
```

```
   Tenure  MonthlyCharge  Bandwidth_GB_Year
0 -1.048746   -0.003943   -1.138487
1 -1.262001    1.630326   -1.185876
2 -0.709940   -0.295225   -0.612138
3 -0.659524   -1.226521   -0.561857
4 -1.242551   -0.528086   -1.428184
```

Out[11]: <matplotlib.legend.Legend at 0x20615c812d0>



6. Extracting the cleaned and scaled data to a CSV file.

I exported the cleaned and scaled data to CSV files.

```
#Extracting cleaned data and scaled data to CSV file
#Cleaned data
features_df.to_csv(OutputfilePath)

# Scaled data
scaled_df.to_csv(Scaledfilepath)
```

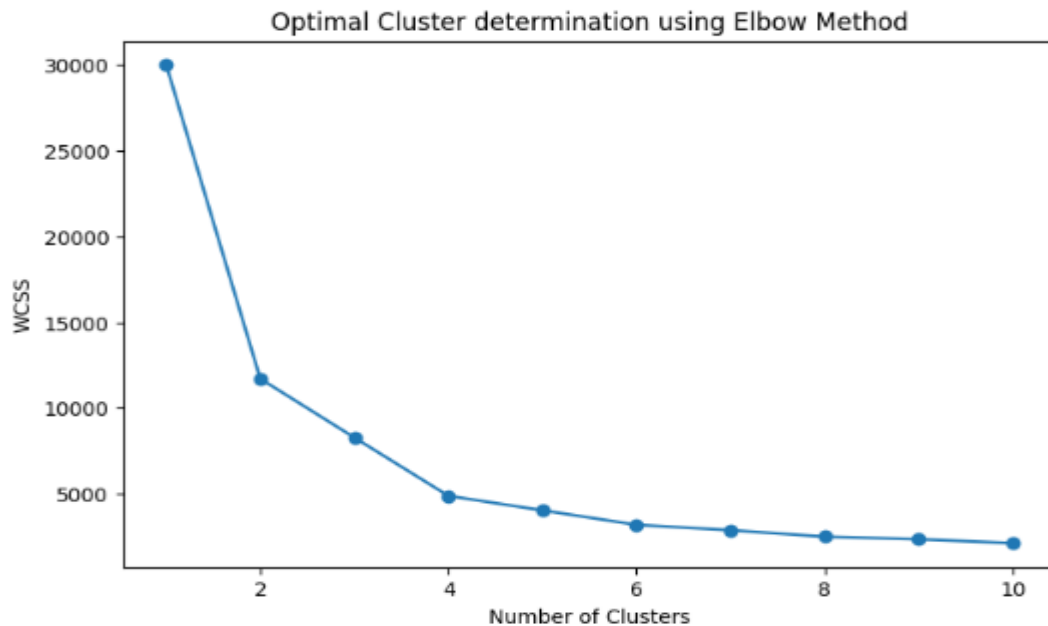
7. Analyzing the dataset using K-Means Clustering

- Determining the number of clusters using the elbow method

```
#Determining the optimal number of clusters (k) using an elbow
wcss = [] # Within-Cluster-Sum-of-Squares

for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42)
    kmeans.fit(scaled_df)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(8,5))
plt.plot(range(1, 11), wcss, marker='o')
plt.title('Optimal Cluster determination using Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
```



- Evaluating the optimal cluster

```
#Evaluating the optimal clusters using silhouette score

#The Silhouette Score is a metric used to evaluate how well clusters are formed in clustering
#algorithms like K-Means. It measures how similar an object is to its own cluster compared to
#other clusters.

for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(scaled_df)
    score = silhouette_score(scaled_df, labels)
    print(f'k={k}, silhouette score={score:.3f}')

k=2, silhouette score=0.564
k=3, silhouette score=0.501
k=4, silhouette score=0.476
k=5, silhouette score=0.433
k=6, silhouette score=0.397
k=7, silhouette score=0.369
k=8, silhouette score=0.370
k=9, silhouette score=0.368
k=10, silhouette score=0.360
```

- Applying K-Means clustering, visualizing and analyzing the results

```
#Apply KMeans Clustering
k_optimal = 2
kmeans_model = KMeans(n_clusters=k_optimal, random_state=42)
Churn_df['Cluster'] = kmeans_model.fit_predict(scaled_df)
```

```
# View the cluster centers (centroids) in scaled form
centroids_scaled = kmeans_model.cluster_centers_
centroids_df_scaled = pd.DataFrame(centroids_scaled, columns=scaled_df.columns)
print("Cluster centroids (scaled features):")
print(centroids_df_scaled)
```

```
Cluster centroids (scaled features):
      Tenure  MonthlyCharge  Bandwidth_GB_Year
0 -0.960316      0.002542      -0.951889
1  0.960700     -0.002543       0.952270
```

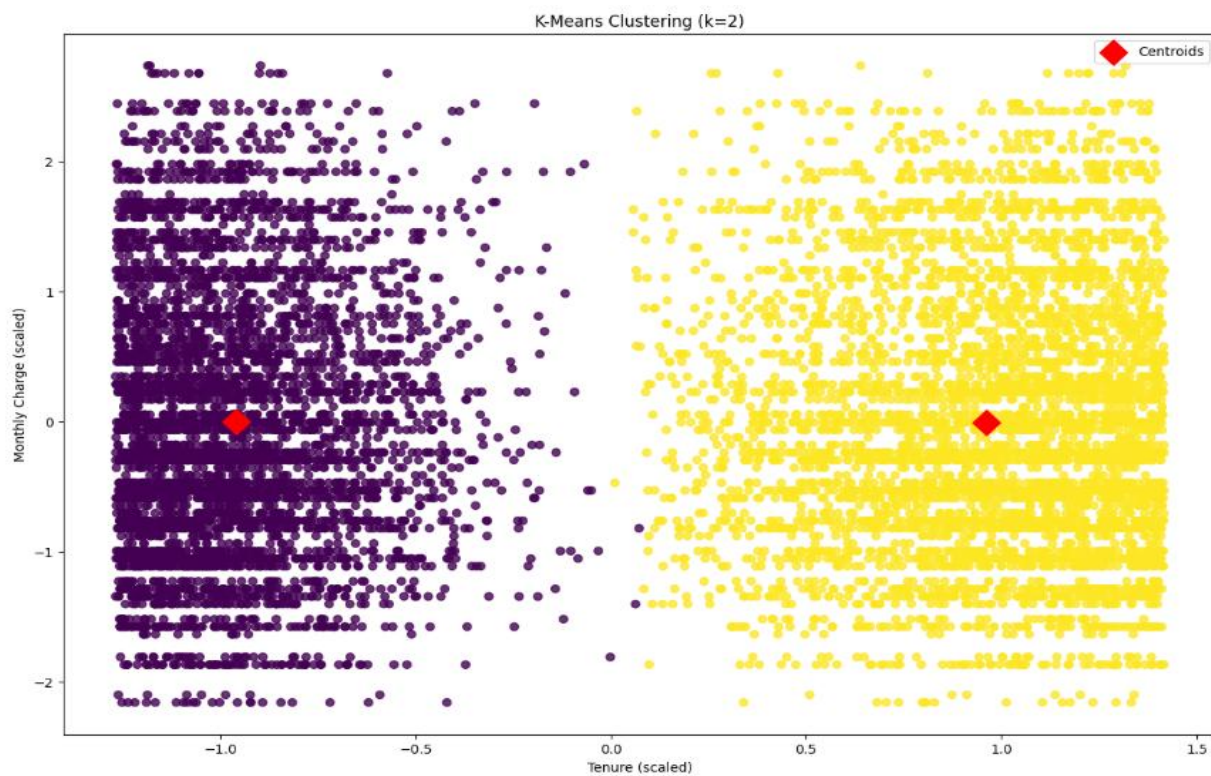
```
# Convert the centroids back to the original scale
centroids_original = scaler.inverse_transform(centroids_scaled)
centroids_df_original = pd.DataFrame(centroids_original, columns=scaled_df.columns)
print("\nCluster centroids (original scale):")
print(centroids_df_original)
```

```
Cluster centroids (original scale):
      Tenure  MonthlyCharge  Bandwidth_GB_Year
0   9.133764    172.733981    1312.287977
1  59.928772    172.515608    5473.227310
```

```
#Summarize Original Data
summary = Churn_df.groupby('Cluster')[['Tenure', 'MonthlyCharge', 'Bandwidth_GB_Year']].mean()
print(summary)
```

```
      Tenure  MonthlyCharge  Bandwidth_GB_Year
Cluster
0      9.133764    172.733981    1312.287977
1     59.928772    172.515608    5473.227310
```

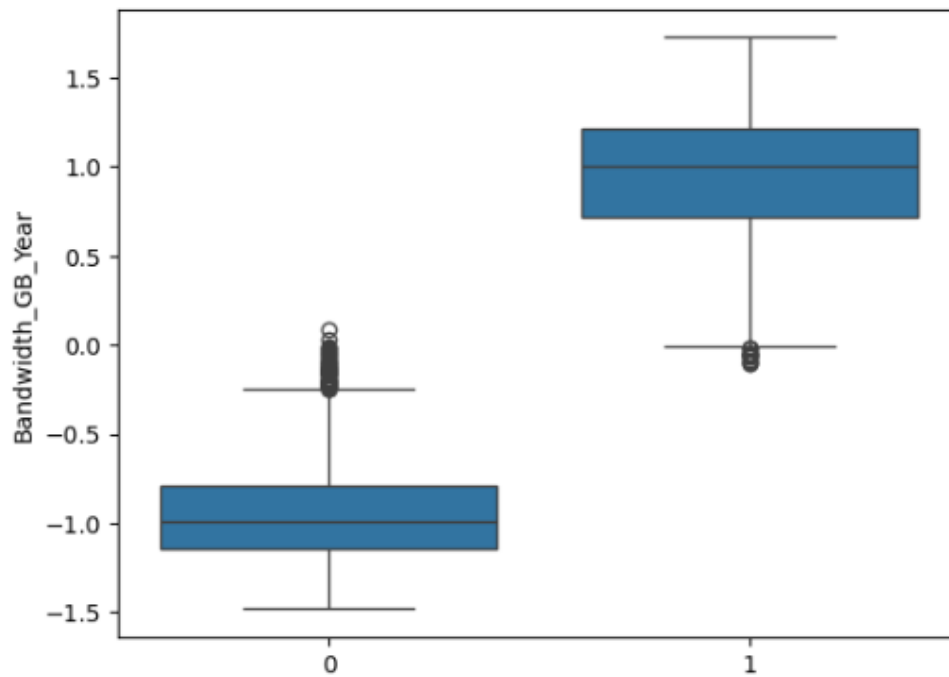
```
# Visualize the clusters using two features to show where the centroids for k_model are located
plt.figure(figsize=(15, 10))
plt.scatter(scaled_df['Tenure'],
            scaled_df['MonthlyCharge'],
            c=Churn_df['Cluster'],
            cmap='viridis', alpha=0.8)
plt.scatter(centroids_scaled[:, 0],
            centroids_scaled[:, 1],
            s=200, c='red',
            marker='D',
            label='Centroids')
plt.xlabel('Tenure (scaled)')
plt.ylabel('Monthly Charge (scaled)')
plt.title('K-Means Clustering (k=2)')
plt.legend()
#plt.colorbar(label='Cluster')
plt.show()
```



```
#Displaying the bandwidth for the two kmeans clusters
```

```
sns.boxplot(x = kmeans_model.labels_, y = scaled_df['Bandwidth_GB_Year'])
```

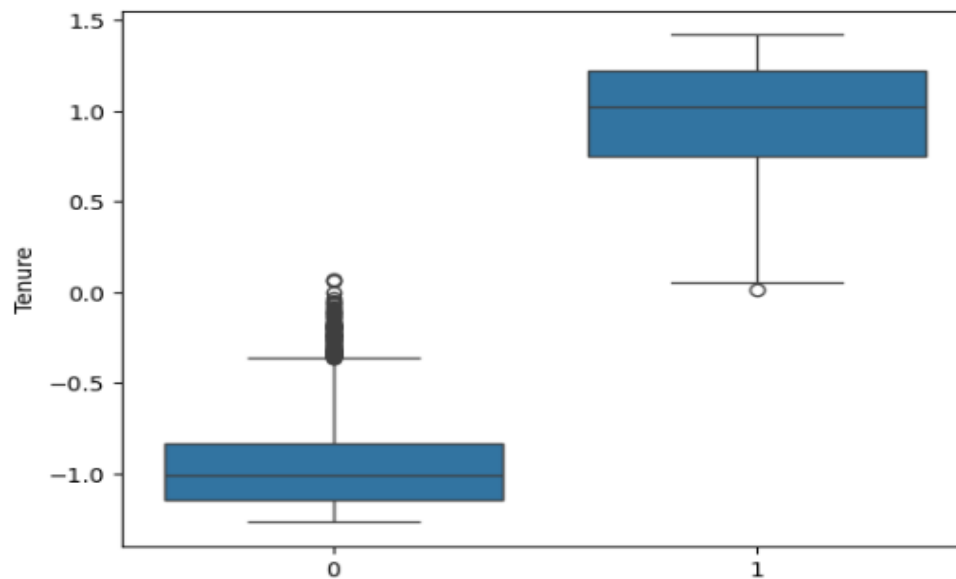
```
<Axes: ylabel='Bandwidth_GB_Year'>
```



```
#Displaying the Tenure for the two kmeans clusters
```

```
sns.boxplot(x = kmeans_model.labels_, y = scaled_df['Tenure'])
```

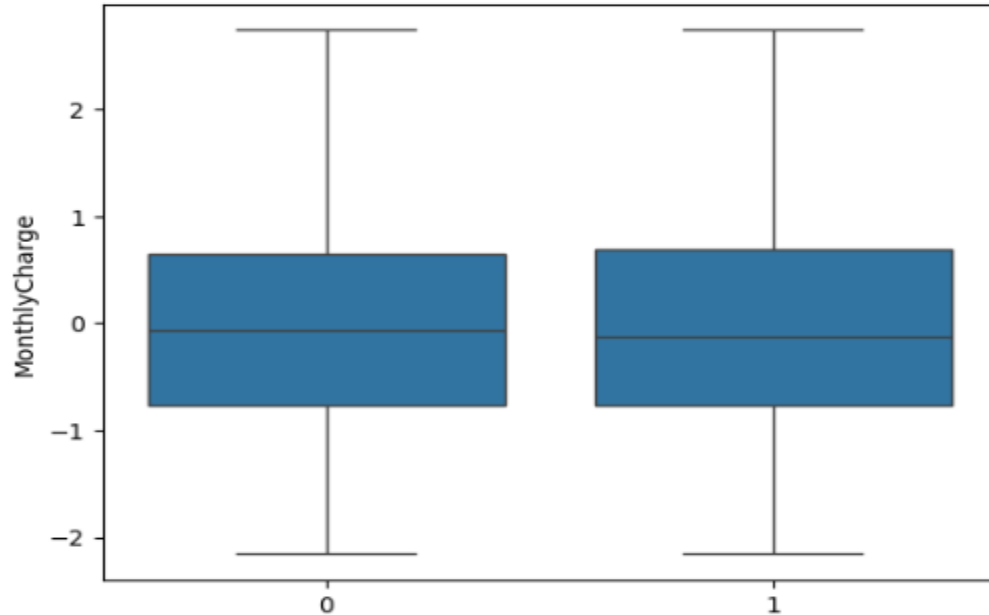
```
<Axes: ylabel='Tenure'>
```



```
#Displaying the Monthly Charge for the two kmeans clusters
```

```
sns.boxplot(x = kmeans_model.labels_, y = scaled_df['MonthlyCharge'])
```

```
<Axes: ylabel='MonthlyCharge'>
```



```
# Evaluate cluster quality using Silhouette Score
```

```
from sklearn.metrics import silhouette_score
```

```
sil_score = silhouette_score(scaled_df, Churn_df['Cluster'])
```

```
print(f"\nSilhouette Score for k=2: {sil_score:.3f}")
```

Silhouette Score for k=2: 0.564

```
Kmeans_inertia = kmeans_model.inertia_
```

```
Kmeans_inertia
```

11709.63194173196

```
# Group by cluster and compute averages
```

```
cluster_profile = Churn_df.groupby('Cluster')[['Tenure', 'MonthlyCharge', 'Bandwidth_GB_Year']].mean()
```

```
# Add count of records per cluster
```

```
cluster_profile['Customercount'] = Churn_df['Cluster'].value_counts()
```

```
print(cluster_profile)
```

	Tenure	MonthlyCharge	Bandwidth_GB_Year	Customercount
Cluster				
0	9.133764	172.733981	1312.287977	5001
1	59.928772	172.515608	5473.227310	4999

C4:CLEANED DATA SET

Copy of the cleaned dataset



Copy of the scaled dataset



D1: OUTPUT AND INTERMEDIATE CALCULATIONS

In order to determine the optimal number of clusters in the dataset, the Elbow Method was used. The elbow method provided the visual validation of the number of clusters to use, whereas the accuracy of the results was statistically validated using the Silhouette Score. This ensured that the K-Means clustering solution is well-grounded and reliable. The two methods independently pointed to 2 clusters as the optimal number for this dataset. This dual confirmation ensures that the final clustering model is both statistically sound and interpretable, providing a strong foundation for further data analysis or business decision-making.

1. Elbow Method

The Elbow Method is a graphical technique used to estimate the ideal number of clusters by evaluating the within-cluster sum of squares (WCSS), a measure of how compact the clusters are. The method involved calculating WCSS for a range of cluster values from 1 to 11 and plotting the results. As expected, the WCSS decreased as the number of clusters increased, but at a diminishing rate. The "elbow" point—where the rate of decrease noticeably slowed—was

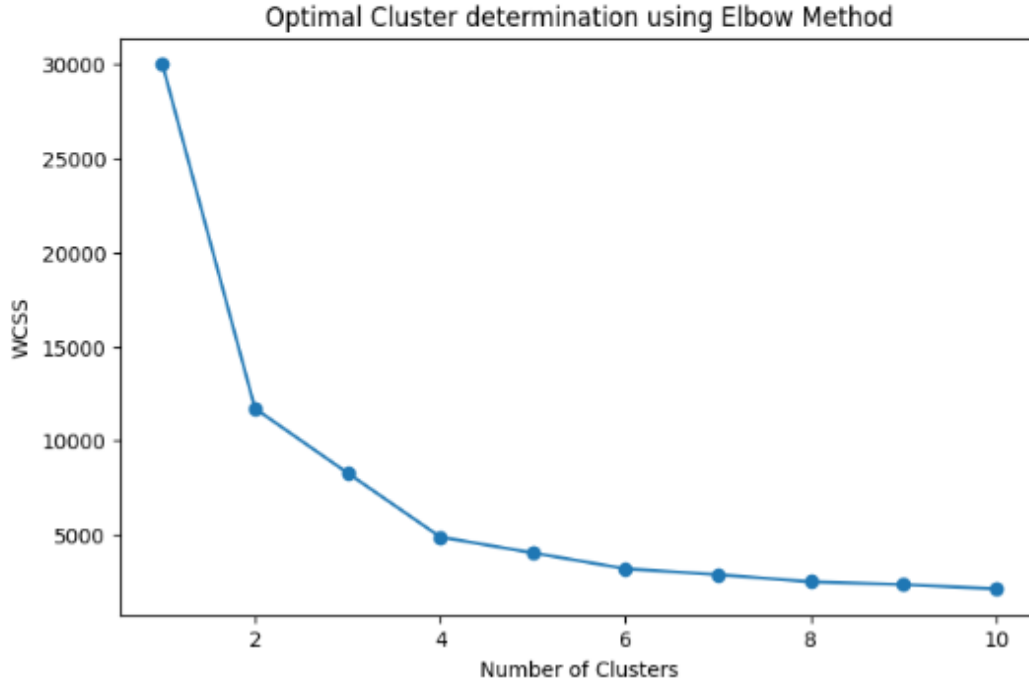
observed at 2 clusters. This indicates that adding more clusters beyond this point yields minimal improvements in intra-cluster cohesion and suggests 2 clusters as the optimal choice.

According to Whitfield, Brennan (2025, March 13), the elbow method is a graphical representation technique used in k-means clustering to find the optimal 'K' value (the number of data clusters partitioned during clustering). This method is typically done by picking out the K value where the elbow (the point where a curve noticeably bends) is created.

```
#Determining the optimal number of clusters (k) using an elbow
wcss = [] # Within-Cluster-Sum-of-Squares

for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42)
    kmeans.fit(scaled_df)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(8,5))
plt.plot(range(1, 11), wcss, marker='o')
plt.title('Optimal Cluster determination using Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
```



2. Silhouette Score

The Silhouette Score is a metric that quantifies how well each data point fits within its assigned cluster compared to other clusters, providing insight into the separation between clusters.

According to Whitfield, Brennan (2025, March 13), the Silhouette score is a very useful method to find the number of K when the elbow method doesn't show a clear elbow point.

Silhouette scores range from -1 to 1, where higher values reflect better clustering. The analysis was performed over a cluster range of 2 to 11. The score was highest when the number of clusters was 2, indicating that this configuration offered the best-defined and most coherent grouping of data points. K=2 has the highest score at 0.564.

```
#Evaluating the optimal clusters using silhouette score

#The Silhouette Score is a metric used to evaluate how well clusters are formed in clustering
#algorithms like K-Means. It measures how similar an object is to its own cluster compared to\
#other clusters.

for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(scaled_df)
    score = silhouette_score(scaled_df, labels)
    print(f'k={k}, silhouette score={score:.3f}')

k=2, silhouette score=0.564
k=3, silhouette score=0.501
k=4, silhouette score=0.476
k=5, silhouette score=0.433
k=6, silhouette score=0.397
k=7, silhouette score=0.369
k=8, silhouette score=0.370
k=9, silhouette score=0.368
k=10, silhouette score=0.360
```

D2: CODE EXECUTION

Please modify the local file path in the code

Local_path = 'C:/Users/beatr/OneDrive/Desktop/WGU/D212/Data/'



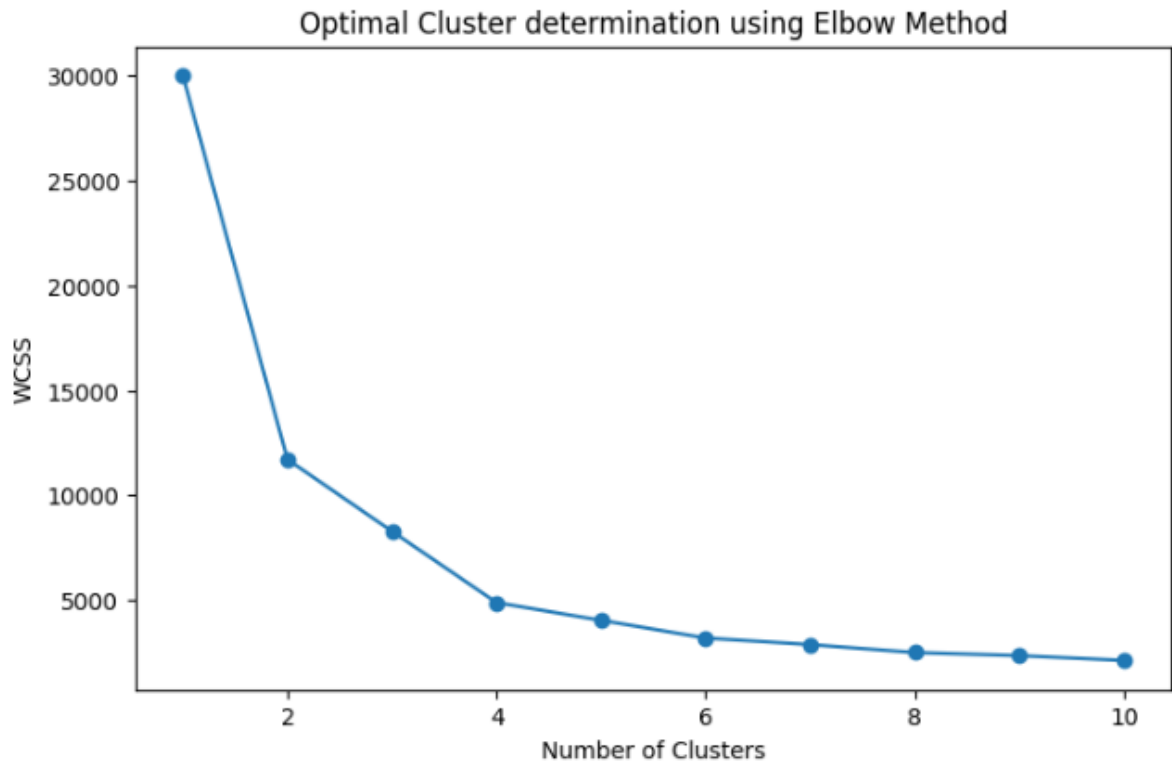
BeatriceMwangi_D2
12 DataMining II_Ta

1: QUALITY OF THE CLUSTERING TECHNIQUE

In assessing the quality of the clusters generated by the K-Means clustering, two key metrics were utilized. The Inertia (Within-Cluster Sum of Squares -WCSS) and the Silhouette score metrics were utilized.

- The Inertia (Within-Cluster Sum of Squares – WCSS)

The Inertia represents the sum of squared distances between each point and the centroid of its assigned cluster. It reflects cluster compactness. According to Diccico, Mathew (2022, February 2), Inertia is the cluster sum of squares. A lower inertia means tighter, more compact clusters, whereas a higher inertia means more spread-out clusters. The inertia decreases as the number of clusters increases, and it is used alongside the Elbow Method to identify the point where adding more clusters yields diminishing returns. The Elbow Method was applied using inertia values across $k = 1$ to 11. The elbow point was observed at $k = 2$, confirming it as the optimal number of clusters. The inertia for $k = 2$ was 11,709.63, indicating a reasonably tight grouping of data points within each cluster.



```
Kmeans_inertia = kmeans_model.inertia_  
Kmeans_inertia
```

11709.63194173196

- Silhouette Score

The Silhouette Score quantifies how similar each data point is to its own cluster compared to other clusters. It reflects both cohesion (how close points in a cluster are) and separation (how far apart clusters are).

The Silhouette score range indicators are

+1 indicates strong clustering

0 indicates borderline clustering

-1 indicates poor clustering

In this K-Means model where $k = 2$, the Silhouette Score was 0.564, indicating moderate-to-good separation and cohesion. This suggests that the clusters are reasonably well-defined, and most points are appropriately grouped.

```
| # Evaluate cluster quality using Silhouette Score
| from sklearn.metrics import silhouette_score
|
| sil_score = silhouette_score(scaled_df, Churn_df['Cluster'])
| print(f"\nSilhouette Score for k=2: {sil_score:.3f}")
```

Silhouette Score for k=2: 0.564

E2: RESULTS AND IMPLICATIONS

The K-Means clustering analysis resulted in two distinct clusters, which represent two distinct customer segments. These clusters highlight meaningful patterns in customer behavior based on Tenure, MonthlyCharge, and Bandwidth_GB_Year.

The customers' profiles are as below

```
# Group by cluster and compute averages
cluster_profile = Churn_df.groupby('Cluster')[['Tenure', 'MonthlyCharge', 'Bandwidth_GB_Year']].mean()

|
| # Add count of records per cluster
| cluster_profile['Customercount'] = Churn_df['Cluster'].value_counts()
| print(cluster_profile)
```

Cluster	Tenure	MonthlyCharge	Bandwidth_GB_Year	Customercount
0	9.133764	172.733981	1312.287977	5001
1	59.928772	172.515608	5473.227310	4999

Cluster 0 represents 5001 customers with lower average tenure, lower average Bandwidth_GB_Year, and slightly higher average MonthlyCharge. On the other hand, cluster 1 represents 4999 customers with longer average tenure, lower average MonthlyCharge, and higher mean Bandwidth_GB_Year.

The implication of this analysis is that we are able to identify the two groups in our telecommunication churn dataset customer base using the K-Means Clustering machine learning method, as shown by the cluster profile above. Based on this, we can seek to formulate retention strategies for customers with low tenure by increasing the Bandwidth_GB_Year or re-evaluating the pricing model across the two cluster groups so as to increase our profitability.

E3: LIMITATION

The K-Means Clustering analysis was limited by a lack of understanding of how the data was collected and domain knowledge. Whereas the clustering results that were successfully generated, the interpretation and formulation of strategic action points were limited by a lack of domain expertise. A deeper insight into the telecommunication churn dataset is needed for further analysis.

E4: COURSE OF ACTION

Based on the results of the K-Means clustering analysis, I recommend that the telecommunication leadership formulate strategies for retaining the customers under cluster 0 by increasing the Bandwidth_GB_Year so as to increase engagement and reduce churn, and also to re-evaluate the pricing model for the customers under cluster 1 which will result in higher profitability for the business.

```
# Group by cluster and compute averages
cluster_profile = Churn_df.groupby('Cluster')[['Tenure', 'MonthlyCharge', 'Bandwidth_GB_Year']].mean()

# Add count of records per cluster
cluster_profile['Customercount'] = Churn_df['Cluster'].value_counts()
print(cluster_profile)
```

Cluster	Tenure	MonthlyCharge	Bandwidth_GB_Year	Customercount
0	9.133764	172.733981	1312.287977	5001
1	59.928772	172.515608	5473.227310	4999

F: PANOPTO VIDEO OF CODE

F1: PANOPTO VIDEO OF PROGRAMS

<https://wgu.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=30bf08c2-6707-4162-aab3-b30e00c60972>

G: SOURCES FOR THIRD-PARTY CODE

None

H: SOURCES

Whitfield, Brennan (2025, March 13) Elbow Method in K-Means Clustering: Definition, Drawbacks, vs. Silhouette Score

<https://builtin.com/data-science/elbow-method>

Dicicco, Mathew (2022, February 2) Inertia for ML Applications

<https://medium.com/@matthew.dicicco38/inertia-for-ml-applications-8c38de2d10d7>

I: PROFESSIONAL COMMUNICATION